HY-225 - Ex. 04-05 - csd5748

Άσκηση 4.2 – Ερώτημα α:

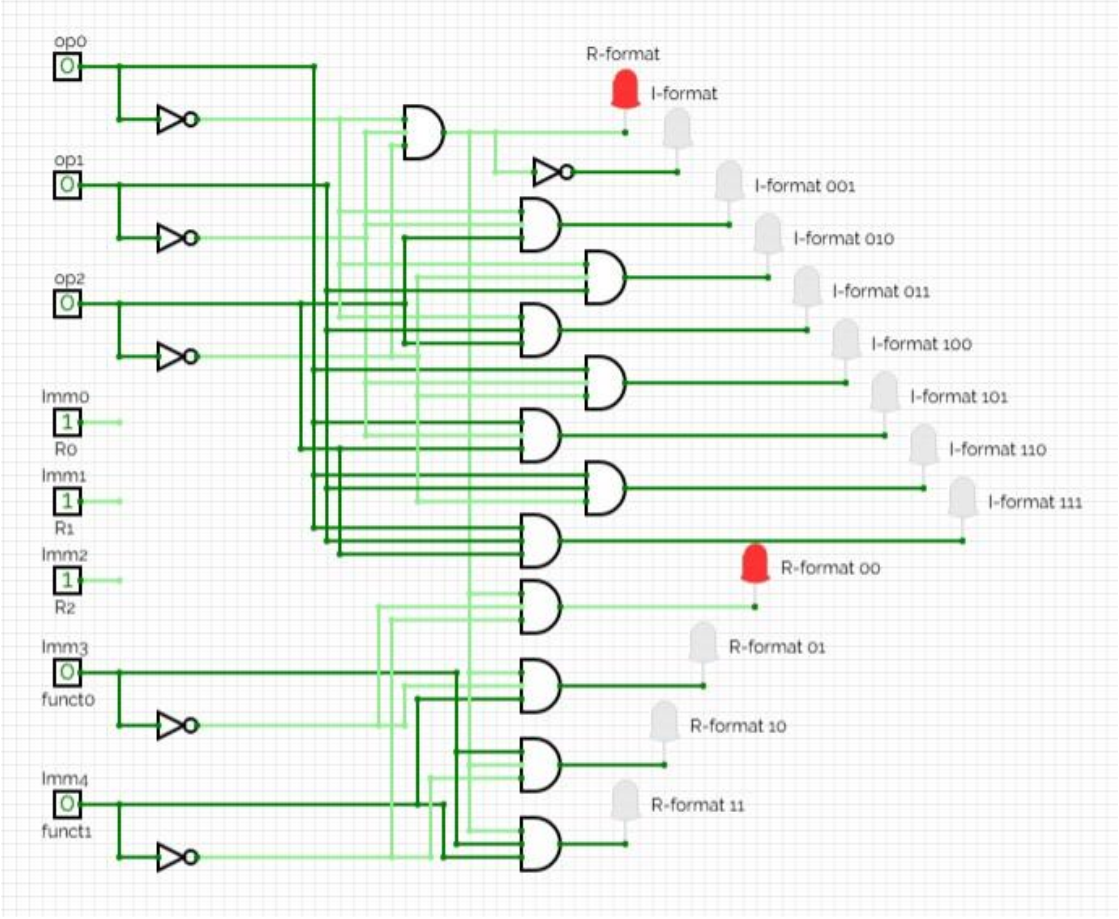
Εάν ξοδέψουμε και τους 8 διαθέσιμους συνδυασμούς του πεδίου op για 8 εντολές I-format (τις εντολές ki, li, mi, ni, pi, qi, ri, si) δεν μπορούμε να έχουμε καμία εντολή R-format. Αυτό συμβαίνει επειδή με τα 3 διαθέσημα bits, μπορούμε να έχουμε 2^3=8 πιθανούς συνδυασμούς (000 έως 111), οι οπίοι όμως χρησιμοποιούνται όλοι. Έτσι, εάν επιμένουμε να έχουμε την εντολή R-format “RR” με op=001 και funct=10, τότε, η εντολή “RR x5” θα μπορούσε να διαβαστεί ως I-format εντολή με op=001 και Imm=10110, άρα την li 22 I-format εντολή, με συνέπεια να μη μπορούμε να τις έχουμε και τις δύο στον RV8. Τέλος, αυτό θα γινόταν με οποιαδήποτε εντολή και αν προσπαθούσαμε να είχαμε αφού κάθε 8 bits μπορούν πάντα να διαβαστούν είτε ως op – Imm στο I-format, είτε ως op – R – funct στο R-format.

Άσκηση 4.2 – Ερώτημα β:

Εάν τώρα ξοδέψουμε μόνο τους 7 από τους 8 διαθέσιμους συνδυασμούς του πεδίου op (τους 001, 010, 011, 100, 101, 110, και 111) για 7 εντολές I-format (τις ki, li, mi, ni, pi, qi, ri), τότε μένει μόνο ένας διαθέσιμος συνδυασμός (ο 000), οπότε θα μπορούμε να έχουμε όσες εντολές μας επιτρέπει ο funct στο R-format. Άρα, αφού funct=2 τότε 2^2=4 εντολές. Έτσι, οι κώδικες op και funct που θα έχει η καθεμιά τους είναι: op=000-funct=00, op=000-funct=01, op=000-funct=10 και op=000-funct=11 (διαδικό). Τέλος, αφού οι υπόλοιποι κώδικες op είναι πιασμένοι από εντολές I-format, δεν μπορούμε να έχουμε παραπάνω εντολές R-format όπως εξηγήθηκε προηγουμένως.

Άσκηση 4.2 – Ερώτημα γ:

Σύμφωνα με το ερώτημα (β), έχουμε 8 bits. Επίσης, το ένα σήμα I θα ανάβει όταν δεν θα ανάβει το ένα σήμα R, αφού η εντολή θα είναι είτε I-format είτε R-format. Έτσι, αφού το σήμα R πρέπει να ανάβει για op=000, το I θα ανάβει για op!=000. Τέλος, θα έχουμε και 7+4=11 σήματα, για όλες τις εντολές.
[Το παρακάτω κύκλωμα δημιουργήθηκε στο circuitverse.org όπου οι πύλες έχουν δύο εισόδους, όμως χρησιμοποιώ τρεις. Έτσι, πρακτικά δεν δουλεύει αλλά η λογική είναι αυτή και πιστεύω πως είναι σωστή!]



Άσκηση 4.2 – Ερώτημα δ:

Εάν τώρα ξοδέψουμε μόνο τους 6 από τους 8 διαθέσιμους συνδυασμούς του πεδίου op (έστω τους 001, 010, 011, 100, 101 και 110) για 6 εντολές I-format (οποιεσδήποτε), τότε μένουν δύο διαθέσιμοι συνδυασμοί (οι 000 και 111), οπότε θα μπορούμε να έχουμε τους διπλάσιες εντολές από πριν, 2*4=8 ή 2*2^2=8 εντολές στο R-format. Έτσι, οι κώδικες op και funct που θα έχει η καθεμιά τους στο R-format είναι: op=000-funct=00, op=000-funct=01, op=000-funct=10, op=000-funct=11 και op=111-funct=00, op=111-funct=01, op=111-funct=10, op=111-funct=11. Και οι κώδικες op που θα έχει η καθεμιά τους στο I-format είναι: op=001, op=010, op=011, op=100, op=101 και op=110.

Άσκηση 5.2 – Ερώτημα α:

Ο RISC-V *δεν* έχει εντολή ble (branch if less or equal), ούτε εντολή bgt (branch if greater than). Αυτό συμβαίνει επειδή δεν χρειάζονται διαφορετικές εντολές για κάθε περίπτωση σύγκρισης αφού υπάρχουν οι 6 κυριότερες (beq, bne, blt, bge, bltu και bgeu) και με τον συνδυασμό τους μπορούν να δημιουργηθούν όλες οι υπόλοιπες. Έτσι, υπάρχουν λιγότερες και πιο απλές εντολές. Έτσι, το i ≤ j για παράδειγμα μπορεί να υπολογιστεί ως: (blt x22, x23, label) και έπειτα (beq x22, x23, label). Επιπλέον, το i > j υπολογίζεται ως: (bge x22, x23, label) και έπειτα (bne x22, x23, label).

Άσκηση 5.2 – Ερώτημα β:

Ο RISC-V δεν έχει τις εντολές bequ (branch if equal unsigned) και bneu (branch if not equal unsigned), επειδή οι εντολές beq (branch if equal) και bne (branch if not equal) που έχει, συγκρίνουν δύο καταχωρητές bit προς bit χωρίς να λαμβάνουν υπόψη αν οι αριθμοί είναι υπογεγραμμένοι (signed) ή όχι (unsigned). Έτσι, αυτές οι εντολές συγκρίνουν ένα-ένα τα bits των δύο καταχωρητών και εξετάζουν αν είναι ίσα ή διαφορετικά, χωρίς να υπολογίζουν τι αριθμοί είναι. Από την άλλη, αν υπήρχαν αυτές οι εντολές, θα σύγκριναν τους αριθμούς σαν θετικούς, χωρίς να λαμβάνουν υπόψη το πρόσημο. Τελικά, το αποτέλεσμα θα είναι το ίδιο, άρα δεν χρειάζονται αυτές οι εντολές.

Άσκηση 5.2 – Ερώτημα γ:

Όλες οι εντολές διακλάδωσης του RISC-V συγκρίνουν πάντα μόνον δύο καταχωρητές μεταξύ τους, δεν υπάρχουν εντολές διακλάδωσης που να συγκρίνουν το περιεχόμενο ενός καταχωρητή με μία σταθερή ποσότητα (όπως π.χ. beqi rs1, Immediate, Label). Όπως είπαμε, αυτό συμβαίνει για λόγους απλότητας στις διαθέσιμες εντολές, αφού δεν χρειάζεται διαφορετική τέτοια εντολή από την στιγμή που απλά μπορούμε να περάσουμε την σταθερή ποσότητα σε έναν καταχωρητή και έπειτα να τον συγκρίνουμε με έναν άλλο.

Άσκηση 5.2 – Ερώτημα δ:

Με βάση τα προηγούμενα ερωτήματα της άσκησης (α και γ), ο ψευδοκώδικας για τον υπολογισμό του "branch if i≤13" θα μεταφραστεί σε Assembly του RISC-V ως:

- addi x26, x0, 13
- blt x22, x26, label
- beq x22, x26, label

Άσκηση 5.5:

```
            add   t1, x0, x0          # i=0;
            j   check54               # jump to condition check
body54:     addi   t1, t1, 1          # i = i+1 (loop BODY)
check54:    slli   t0, t1, 3          # tmp = 8 * i (start COND evaluation)
            add   t0, s0, t0              # tmp = διεύθυνση του table[i]
               ld   t0, 0(t0)             # tmp = table[i]
            bne   t0, t2, body54   # if table[i] != key goto BODY
cont54:     ....                          # continue with the rest of the program
```

Άσκηση 5.11:

Ξέρουμε: x22 --> i και x23 --> j και x26 --> B και έχουμε:

i. B = (i < j): slt x26, x22, x23

ii. B = (i >= j): slt x26, x22, x23 - xori x26, x26, 1

iii. B = (i > j): slt x26, x23, x22

iv. B = (i <= j): slt x26, x23, x22 - xori x26, x26, 1

v. B = (i < CONST): slti x26, x22, CONST

vi. B = (i >= CONST): slti x26, x22, CONST - xori x26, x26, 1

vii. B = (i > CONST): slti x26, CONST, x22

viii. B = (i <= CONST): slti x26, CONST, x22 - xori x26, x26, 1

i. B = (i == j): sub x26, x22, x23 - sltiu x26, x26, 1

ii. B = (i != j): sub x26, x22, x23 - sltu x26, x0, x26

iii. B = (i == CONST): sub x26, x22, CONST - sltiu x26, x26, 1

iv. B = (i != CONST): sub x26, x22, CONST - sltu x26, x0, x26

ΤΕΛΟΣ

(ευχαριστώ)